

# AI 应用开发学习手册

从零到能投简历的完整记录 整理时间：2026-09-17 学习周期：2026-09-10 ~ 2026-09-17（7 个学习日）

## 目录

- 起点与终点
- 学习时间线
- 知识体系 - 阶段一：Agent 基础 - 阶段二：RAG 全链路 - 阶段三：MCP 协议 - 阶段四：服务化 - 阶段五：框架与工程化
- 踩坑录
- 核心概念速查表
- 面试问答准备
- 文件清单
- 还没学的

## 一、起点与终点

| 对比维度   | 起点（09-10） | 终点（09-17）                    |
|--------|-----------|------------------------------|
| Python | 写不出 3 行   | 能写 250 行的 agent              |
| JSON   | 不知道是什么    | 天天在用                         |
| 大模型    | 只会聊天      | 会调 API、会 Function Calling    |
| 项目     | 无         | 一个完整的 RAG 系统（Gitee + GitHub） |

|                                                                                                                               |            |               |
|-------------------------------------------------------------------------------------------------------------------------------|------------|---------------|
| 对比维度                                                                                                                          | 起点 (09-10) | 终点 (09-17)    |
| 简历                                                                                                                            | 空白         | 2 页，带项目链接和证件照 |
| 7 天里，独立完成了： - 一个中文分词器（没用 jieba） - 一套 TF-IDF 检索（没用向量库） - 一个 MCP Server（没用 SDK） - 一个 FastAPI 服务 - 一个 28 题的检索评测 - 一个带可观测性的 Agent |            |               |

---

## 二、学习时间线

| 日期    | 学了什么                              | 产出文件                                                     |
|-------|-----------------------------------|----------------------------------------------------------|
| 09-10 | Agent 是什么——感知→规划→行动→观察的循环         | agent_demo.py                                            |
| 09-12 | 死循环的威力、第一次调 LLM API、对话记忆、工具调用     | loop_demo.py hello_ai.py chat.py agent.py                |
| 09-13 | 文档提取、文本清洗、语义切块                    | build_corpus.py clean_text.py chunk_docs.py              |
| 09-14 | 读 JSON 数据、统计分析                    | stats.py                                                 |
| 09-15 | 中文分词器、TF-IDF 检索、RAG 全链路、模块拆分、效果评测 | search.py index.py rag.py retrieval.py eval_retrieval.py |
| 09-16 | MCP 协议（从零实现）、FastAPI 服务化          | mcp_server.py api.py                                     |
| 09-17 | LangGraph 状态图、可观测性埋点、简历与文档        | langgraph_*.py agent_v2.py                               |

---

## 三、知识体系

### 阶段一：Agent 基础

#### 1.1 调大模型 API —— 最小的一步

```
payload = {  
    "model": "deepseek-chat",  
    "messages": [{"role": "user", "content": "你好"}],  
}  
  
req = urllib.request.Request(  
    "https://api.deepseek.com/chat/completions",  
    data=json.dumps(payload).encode("utf-8"),  
    headers={  
        "Content-Type": "application/json",  
        "Authorization": "Bearer " + API_KEY,  
    },  
)
```

**核心认知：** - API key 从**环境变量**读，不写死在代码里（否则 push 到 GitHub 就泄露） - 请求是 POST，body 是 JSON - 响应也是 JSON，回答在 `choices[0].message.content`

#### 1.2 「记忆」是怎么实现的

**关键认知：大模型没有记忆。**

所谓"多轮对话"，是**每次请求都把完整历史重新发一遍**：

```
messages = [  
    {"role": "user", "content": "我叫张三"},  
    {"role": "assistant", "content": "你好张三"},  
    {"role": "user", "content": "我叫什么? "},      # ← 这一轮  
]  
  
# 整个列表一起发给模型，它「看到」了前面的对话
```

**推论：** - 对话越长，输入 token 越多，越贵越慢 - 这就是「上下文管理」为什么重要

### 1.3 工具调用 (Function Calling)

**核心认知：** AI 只负责"决定"，你的程序负责"执行"。

① 你把工具清单发给模型 (TOOLS\_SCHEMA)

↓

② 模型返回: {"tool\_calls": [{"function": {"name": "get\_time", "arguments": "{}"}}]}

↓

③ 你的代码真的执行 get\_time()

↓

④ 把结果作为 {"role": "tool", ...} 追加进 messages，再发给模型

↓

⑤ 模型基于结果生成最终回答

**加一个工具要改三处：**

| # | 位置           | 改什么             |
|---|--------------|-----------------|
| ① | TOOLS_SCHEMA | 声明工具 (名字、描述、参数) |
| ② | 函数区          | 写实现             |
| ③ | TOOLS 字典     | 注册名字 → 函数       |

**关键：** **description** 决定了模型会不会正确选择这个工具。

### 1.4 Agent 循环

```
while True:

    msg = call_ai(messages)      # 问模型

    messages.append(msg)

    if not msg.get("tool_calls"): # 没有工具调用 = 给出最终答案

        break
```

```
for call in msg["tool_calls"]: # 有工具调用 = 执行

    result = run_tool(...)

    messages.append({"role": "tool", ...})
```

**必须有的防护：**

```
rounds += 1

if rounds > 10:

    print("⚠ 超过 10 轮，强制停止")

    break
```

**为什么？** 模型可能陷入"调工具 → 不满意 → 再调"的死循环。每一轮都是一次 API 调用，都是钱。

## 1.5 工具失败怎么办

**不要让它崩溃，把错误信息回传给模型。**

```
def read_file(path):

    try:

        with open(path, encoding="utf-8") as f:

            return f.read()

    except Exception as e:

        return "读取失败：" + str(e) # ← 当成正常结果返回
```

**模型看到错误信息后，会自己解释原因并给建议。**

实测效果（文件路径 `~` 没展开时）：

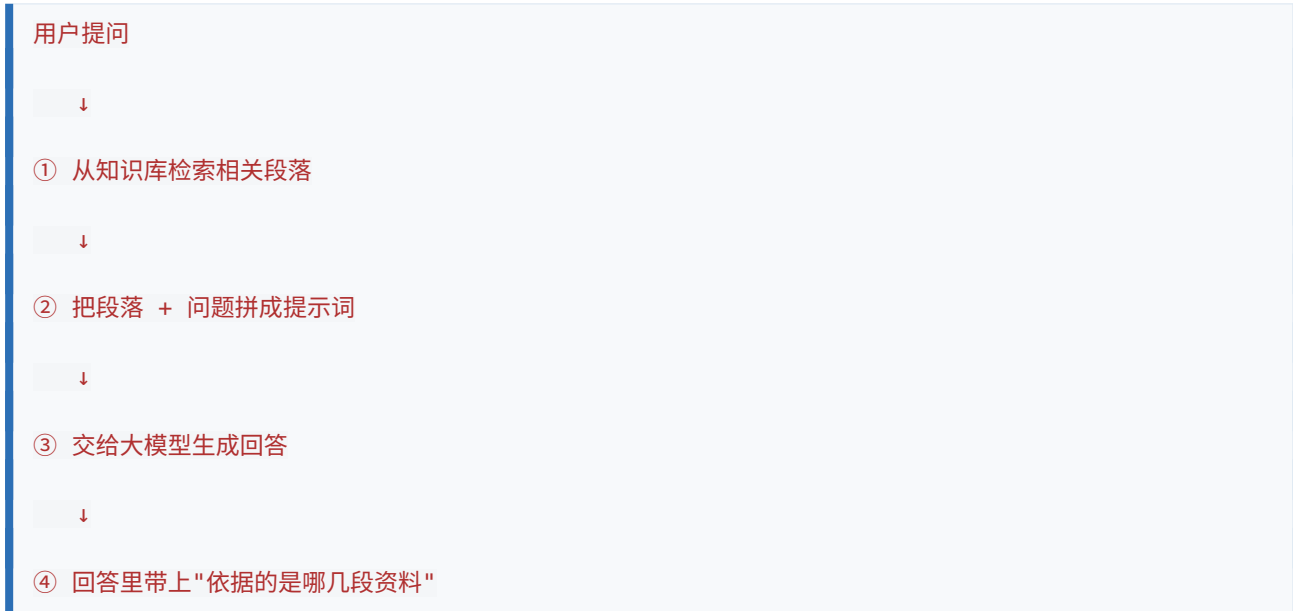
```
"读取失败了，原因是路径中的 ~ 没有被展开。~ 是 shell 的简写，但文件读取工具是按字面路径去找的……"
```

## 阶段二：RAG 全链路

### 2.1 为什么要 RAG

**问题：**大模型不知道你的私有资料。直接问它，它会瞎编。

**RAG 的思路：**先从你的资料里检索相关段落，再让模型"看着资料回答"。



### 2.2 ① 文档提取

**工具：**`pdftotext`（命令行）+ Python 标准库

| 格式  | 方法                             |
|-----|--------------------------------|
| PDF | <code>pdftotext -layout</code> |
| doc | <code>zipfile</code> 解压 + 解析   |
| x   | <code>word/document.xml</code> |

### 2.3 ② 清洗修复

**PDF 提取出来是脏的，必须修：**

| 问题   | 现象                                   | 解法     |
|------|--------------------------------------|--------|
| 标题被吞 | 密码皆为：ChinaSkills23。1. DCserver 系统... | 正则重新断行 |

| 问题    | 现象                         | 解法                                    |
|-------|----------------------------|---------------------------------------|
| 换页符作祟 | 不可见的 <code>\f</code> 被当成缩进 | <code>text.replace("\f", "\n")</code> |
| 目录污染  | 目录含所有标题，任何查询都命中它           | 识别并整体剔除                               |

正则断行的陷阱（重点）：

```
BURIED_HEADING = re.compile(r"([。；])\s*([0-9]+[.、.](?=[^\d]))")
```

那个 `(?=[^\d])` 负向断言是关键：

```
172.16.100.221; 8.8.8.8; 5. 为 ChinaSkills.cn 域配置安全策略
```

↑                    ↑

IP 地址            这才是标题

没有这个断言，82 个 IP 地址会被切得七零八落。

## 2.4 ③ 切块策略

为什么不能整篇丢进去？上下文长度有限 + 检索不精确。

方案：按标题层级切，每块带上「面包屑」。

```
模块 B：服务部署 > Linux 项目任务清单 > 服务器 RouterSrv 上的工作任务
```

面包屑的作用：保持上下文。单独看一块可能不知道它在讲什么，带上路径就清楚了。

结果：4 万字语料 → 131 个知识块，平均 274 字。

## 2.5 ④ 中文分词（自己实现的）

中文没有空格，必须分词。

方案：二元切分（bigram）——每相邻两个字组成一个词。

```
"中继怎么配" → 中继 | 继怎 | 怎么 | 么配
```

✓            ✓            □□

中间那两个"噪音词"无害——它们在知识库里不出现，权重自然是 0。

**关键陷阱：ASCII 部分不能被切开。**

```
def char_type(ch):  
    if '\u4e00' <= ch <= '\u9fff':  
        return "cn"  
  
    if ch.isascii() and (ch.isalnum() or ch == "."):  
        return "ascii"      # ← 小数点必须算进来  
  
    return "other"
```

**为什么？** 172.16.100.221 如果按字符切，会变成 172/16/100/221 四段，彻底失去意义。

**流程：**按字符类型分组 → 中文段做 bigram → ASCII 段整体保留。

## 2.6 ⑤ TF-IDF 加权检索

**朴素做法的问题：**命中一个词加 1 分 → 常见词把分数"灌水"。

问题："怎么配置 WSUS 更新服务"

分词：['怎么', '么配', '配置', 'WSUS', '更新', '新服', '服务']

↑ 出现于 74 个块      ↑ 只出现于 2 个块

毫无区分度                      极强区分度

**IDF 加权：**一个词出现的块越少，它越有价值。

权重 = 总块数 ÷ 出现该词的块数

WSUS →  $131 / 2 = 65.5$

配置 →  $131 / 74 = 1.8$       ← 相差 36 倍

**效果：Top-1 与次优结果的分差从 2:1 拉开到 7:1。**

## 2.7 ⑥ 幻觉防护

**三条硬规则写进提示词：**

1. 只能使用资料中的内容，不要使用你自己的知识
2. 如果资料中没有答案，直接回答「资料里没有找到相关内容」

3. 回答末尾标注依据的是哪几条资料

实测验证：

| 问题             | 结果                  |
|----------------|---------------------|
| DHCP 中继怎么配     | ✓ 检索到相关块，回答并标注来源    |
| 今天天气怎么样        | ✓ 正确回答"资料里没有找到相关内容" |
| 怎么用 Python 写爬虫 | ✓ 正确拒答              |

一个会说"我不知道"的系统，比一个乱编的系统可靠得多。

2.8 ⑦ 效果评测（最有价值的一步）

方法：构建测试集，人工标注正确答案应包含的关键词。

两组对比：

| 组别       | 说明                      | Top-1 | Top-3 |
|----------|-------------------------|-------|-------|
| A 组：直接提问 | 用原文术语 ("DHCP 中继怎么配")    | 100%  | 100%  |
| B 组：改写提问 | 用同义表达 ("怎么让客户端自动获取 IP") | 75%   | 75%   |

两个失败案例暴露了真问题：

| 失败的问题         | 期望答案     | 根因                       |
|---------------|----------|--------------------------|
| 怎么限制员工上班时间玩游戏 | P2P 应用阻断 | 语料原文是"P2P 应用软件"，没有"游戏"一词 |
| 怎么让外网访问内网服务   | NAT 配置   | 语料原文是"内部客户端访问互联网"        |

结论：关键词检索只能匹配字面，对同义表达完全无能。这是固有局限，需要向量检索（Embedding）解决。

"我的系统很准"和"我知道我的系统哪里不准、为什么不准"，后者才是工程能力。

阶段三：MCP 协议

3.1 MCP 解决什么问题

没有 MCP (M×N 问题)：

Claude Desktop 想用你的工具 → 写一套对接

Cursor 想用你的工具 → 再写一套

Cline 想用你的工具 → 再写一套

有 MCP (M+N 问题)：

把工具做成「符合 MCP 标准的 Server」

↓

任何支持 MCP 的客户端都能直接调用

类比：MCP 就是 AI 工具界的「插座标准」。

3.2 和 Function Calling 什么关系（面试必问）

| 对比维度 | Function Calling | MCP              |
|------|------------------|------------------|
| 属于哪层 | 模型的能力            | 应用之间的协议          |
| 解决什么 | 模型如何表达"我要调工具"    | AI 应用如何发现和连上外部工具 |
| 谁定的  | 各模型厂商            | 开放的公共协议          |

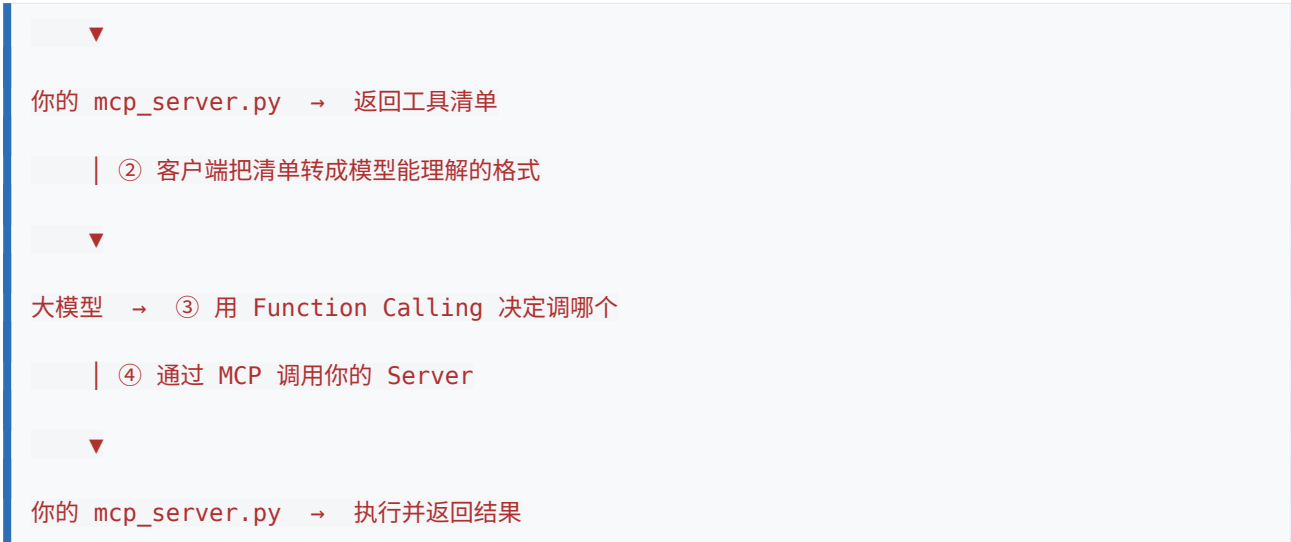
一句话：

MCP 负责"把工具接进来"，Function Calling 负责"让模型决定用哪个"。

完整链路：

Claude Desktop (MCP 客户端)

| ① 通过 MCP 问：你有哪些工具？



3.3 协议本质：就三件事

- ① 从 `stdin` 读一行 JSON
- ② 看 `method` 字段，决定回什么
- ③ 往 `stdout` 回一行 JSON

245 行的完整实现，剥掉细节就这三件事。

3.4 消息格式：JSON-RPC 2.0

| 类型 | 有 id 吗     | 要不要回复 |
|----|------------|-------|
| 请求 | ✓ 有        | 必须回   |
| 响应 | ✓ 带上同一个 id | —     |
| 通知 | ✗ 没有       | 不回复   |

3.5 三个核心方法

| 方法                      | 作用        |
|-------------------------|-----------|
| <code>initialize</code> | 握手，交换能力声明 |

| 方法 | 作用 |
|----|----|
|----|----|

(capabilities)

tools/  
list      返回工具清单 (含 inputSchema)

tools/  
call      执行工具, 结果放在 content 数组里

**注意:** MCP 的字段是 `inputSchema`, 不是 Function Calling 的 `parameters`。

### 3.6 两个实现陷阱

#### ① 必须 `flush()`:

```
sys.stdout.write(json.dumps(msg) + "\n")
sys.stdout.flush()      # ← 不加这个会死锁
```

**为什么?** 输出会先进缓冲区。客户端等回复、服务端等缓冲区满 → 双方互等。

#### ② 中文要 `ensure_ascii=False`:

```
json.dumps(msg, ensure_ascii=False)
# 不加的话中文会变成 \u83b7\u53d6 这种
```

### 3.7 工具分发用字典而非 if/elif

```
TOOL_HANDLERS = {
    "search_knowledge_base": call_search_knowledge_base,
    "get_stats": call_get_stats,
}

handler = TOOL_HANDLERS.get(name)
```

**为什么?**

- `if/elif` 写法: 加工具要**修改**判断逻辑 (插入分支)
- 字典写法: 加工具只要**追加**一行

**这就是「对扩展开放, 对修改关闭」——加代码优于改代码。**

阶段四：服务化（FastAPI）

4.1 为什么要 API

| 对比 维度        | 命令行<br>序  | HTTP<br>API |
|--------------|-----------|-------------|
| 谁能用          | 只有你自<br>己 | 任何程序        |
| 需要装依赖吗       | 必须        | ✗ 不需要       |
| 需要懂你的代码<br>吗 | 必须        | ✗ 不需要       |

API = 把你的能力变成"一个网址"。

4.2 在大模型场景下的特殊意义

API key 必须在服务端：

```
前端直接调大模型：

fetch("https://api.deepseek.com/...", {
  headers: {"Authorization": "Bearer sk-真 key"}  ← F12 就能看到
})

✅ 正确架构：
前端 → 你的后端 API → DeepSeek API
      (key 只在后端)
```

这就是"后端 API"存在的核心理由之一——它是一道守门人。

4.3 接口定义

```
@app.get("/search") # GET: 参数在 URL 里

def search(q: str = Query(..., description="关键词")):
```

```

...

@app.post("/ask") # POST: 参数在 body 里
def ask(req: AskRequest):
    ...

```

为什么 **/ask** 用 **POST**? - 问题可能很长 (URL 有长度限制) - URL 会被服务器日志记录

#### 4.4 Pydantic 模型

```

class AskRequest(BaseModel):
    question: str # 必填
    top_k: int = 3 # 选填

```

FastAPI 自动做的事:

| 客户端发的内容                                        | FastAPI 自动返回       |
|------------------------------------------------|--------------------|
| <code>{}</code>                                | 422 field required |
| <code>{"question": 123}</code>                 | 422 类型错误           |
| <code>{"question": "x", "top_k": "abc"}</code> | 422 类型错误           |

一行校验代码都不用写。

#### 4.5 错误处理

```

try:
    answer = call_llm(prompt)
except Exception as e:
    raise HTTPException(status_code=502, detail="调用大模型失败: " + str(e))

```

| 不处理                               | 处理了        |
|-----------------------------------|------------|
| 500 Internal Server Error, 不知道哪错了 | 502 + 具体原因 |

4.6 自动生成的交互式文档

访问 `/docs` 就能在浏览器里测试所有接口——这是 FastAPI 最爽的地方。

4.7 部署地址的区别

|                |                  |
|----------------|------------------|
| 127.0.0.1:8000 | → 只有本机能访问        |
| 0.0.0.0:8000   | → 监听所有网卡，局域网都能访问 |

阶段五：框架与工程化

5.1 LangGraph：把循环画成图

核心认知：LangGraph 的「状态合并」就是 Python 字典的 `update()`。

```
state = {"name": "无名", "score": 50}

patch = {"name": "小明"}          # 节点返回的「补丁」

state.update(patch)

# → {"name": "小明", "score": 50}    ← name 被覆盖, score 没动
```

规则：

|             |        |
|-------------|--------|
| 补丁里写了什<br>么 | 框架怎么处理 |
|-------------|--------|

某字段有新值      按该字段的**合并器**合并

没提某字段      **保持原样**

是个空字典 `{}`      **什么都不改**

**合并器（Reducer）** 决定"有新值"时是覆盖还是追加：

|     |    |    |
|-----|----|----|
| 合并器 | 效果 | 适合 |
|-----|----|----|

不指定（默      覆      `name`、`score` 这类"当前

|     |    |    |
|-----|----|----|
| 合并器 | 效果 | 适合 |
|-----|----|----|

认)      盖      值"

`append_list`      追加      对话历史这类"累积"的数据

这解释了为什么 agent 的 `messages` 字段要写成:

```
messages: Annotated[list, add_messages]
```

因为对话历史必须累积，如果默认覆盖，新一轮会把前面的全冲掉。

5.2 五个核心概念

| 概念      | 是什么           | 对应手写 agent                         |
|---------|---------------|------------------------------------|
| State   | 声明状态字段        | <code>messages = []</code>         |
| Node    | 函数：读状态 → 返回补丁 | 循环里的一段代码                           |
| Edge    | 决定下一步去哪       | 顺序执行                               |
| Reducer | 补丁怎么合并        | <code>messages.append(...)</code>  |
| 条件边     | 根据判断决定去哪      | <code>if...continue / break</code> |

5.3 条件边

```
def 判断下一步(state):  
    last = state["messages"][-1]  
    if last.get("tool_calls"):  
        return "tools"      # 返回「标签」  
    return "end"  
  
graph.add_conditional_edges("model", 判断下一步, {
```

```

    "tools": "tools",          # 标签 → 目标节点

    "end": END,

})

```

就是一个 **if...else** 换个写法。

#### 5.4 完整 ReAct 图

```

__start__ → model

model → __end__ [条件边: 模型说完了]

model → tools [条件边: 模型要调工具]

tools → model ← 这条边形成循环

```

#### 5.5 可观测性

**为什么要做：** agent 是黑盒，出问题只能猜。

**核心：** 一个记录器 + 4 处埋点。

```

class Trace:

    def add(self, action, detail, duration, usage=None):

        """每发生一件事，记一笔"""

```

| 埋点位置 | 记什么 |
|------|-----|
|------|-----|

|      |      |
|------|------|
| 调模型前 | 开始时间 |
|------|------|

|      |               |
|------|---------------|
| 调模型后 | 耗时 +<br>token |
|------|---------------|

|            |         |
|------------|---------|
| 执行工具前<br>后 | 耗时 + 结果 |
|------------|---------|

|      |    |
|------|----|
| 任务结束 | 汇总 |
|------|----|

**关键改动：** **call\_ai** 要把 **usage** 一起返回。

```

return data["choices"][0]["message"], data.get("usage", {})

```

**费用计算：**

$$\text{费用} = \text{输入 token}/1\text{e}6 \times \text{输入价} + \text{输出 token}/1\text{e}6 \times \text{输出价}$$

5.6 实测发现的三个规律

| 规律              | 数据               | 意义                   |
|-----------------|------------------|----------------------|
| 工具执行几乎不耗时       | 0.00s            | 性能优化的战场只有一个：减少模型调用轮数 |
| 输入 token 远大于输出  | 847:44 (19:1)    | 主要来自工具定义，所以工具描述不能写太长 |
| 输出 token 决定生成时间 | 约 50-90 tokens/秒 | 让模型说短点，既省钱又省时间       |

四、踩坑录

环境类

| # | 坑                           | 根因                      | 解法                                        |
|---|-----------------------------|-------------------------|-------------------------------------------|
| 1 | pip / git 报 ProxyError      | 环境变量里有指向死代理的 http_proxy | unset http_proxy https_proxy ...          |
| 2 | 从 PyPI 下载慢到超时               | 直连国外源                   | 配清华镜像 pypi.tuna.tsinghua.edu.cn           |
| 3 | UnicodeEncodeError: latin-1 | API key 里混进了中文（占位符没替换）  | 加 key 自检，检查非 ASCII 字符                     |
| 4 | 虚拟环境 ensurepip 缺失           | 没装 python3-venv         | sudo apt install python3-pip python3-venv |

代码类

| # | 坑            | 根因   | 解法                     |
|---|--------------|------|------------------------|
| 5 | PDF 标题被吞进上一段 | 排版缺陷 | 正则断行 + (?=[^\d]) 排除 IP |

| #  | 坑                       | 根因                                     | 解法                                    |
|----|-------------------------|----------------------------------------|---------------------------------------|
| 6  | 面包屑被污染                  | 不可见的换页符 <code>\f</code> 伪装成缩进          | <code>text.replace("\f", "\n")</code> |
| 7  | 任何查询都命中目录块              | 目录含所有标题                                | 识别并剔除 TOC                             |
| 8  | IP 地址被切碎                | 中文分词时按字符切了                             | ASCII 段整体保留（含小数点）                     |
| 9  | 检索排序不准                  | 常见词把分数灌水                               | IDF 加权                                |
| 10 | 文件路径 <code>~</code> 读不到 | <code>open()</code> 不认识 <code>~</code> | <code>os.path.expanduser(path)</code> |
| 11 | 测试误判检索失败                | 测试关键词写错了（"Windows 防火墙"多了个空格）           | 验证脚本自己也要验证                            |

## 工具类

| #  | 坑                          | 根因                                            | 解法                                           |
|----|----------------------------|-----------------------------------------------|----------------------------------------------|
| 12 | git 提交者显示"你的名字"            | 占位符没替换                                        | <code>git config + filter-branch</code> 重写历史 |
| 13 | <code>git push</code> 认证失败 | 用户名写错（tianxia vs tianxiawo）                   | 改用 SSH 密钥                                    |
| 14 | PDF 出现孤行                   | LibreOffice 不认 <code>page-break-inside</code> | 调整内容长度控制断点                                   |

## 五、核心概念速查表

### Python ↔ C 对照

| Python              | C                    |
|---------------------|----------------------|
| <code>open()</code> | <code>fopen()</code> |

Python

C

|                            |                           |
|----------------------------|---------------------------|
| <code>json.load()</code>   | 手写解析                      |
| <code>list.append()</code> | <code>arr[n++] = x</code> |
| 无 <code>++</code>          | <code>i++</code>          |
| <code>for x in list</code> | 计数 for 循环                 |
| <code>dict</code>          | 哈希表                       |
| <code>try/except</code>    | 错误码返回                     |

大模型应用核心概念

| 概念               | 一句话                             |
|------------------|---------------------------------|
| Function Calling | 模型决定调哪个工具，你的程序负责执行              |
| messages 列表      | 大模型没有记忆，每轮都重发完整历史               |
| RAG              | 先检索资料，再让模型看着资料回答                |
| 幻觉防护             | 提示词里明确"没有就说没有" + 标注来源           |
| MCP              | 标准化"AI 应用如何连上外部工具"的协议           |
| Agent 循环         | 调模型 → 有工具调用就执行 → 再调模型 → 直到无工具调用 |
| 可观测性             | 记录每步的耗时和 token，把黑盒变白盒           |

关键数字（项目实测）

| 数字  | 含义    |
|-----|-------|
| 131 | 知识块数量 |

| 数字    | 含义                  |
|-------|---------------------|
| 274 字 | 平均块长                |
| 4 万字  | 原始语料规模              |
| 100%  | 直接提问 Top-3 召回率      |
| 75%   | 改写提问 Top-3 召回率      |
| 7:1   | Top-1 与次优结果的分差（优化后） |
| 19:1  | 输入 / 输出 token 比例    |
| 0.00s | 工具执行耗时（可忽略）         |

---

## 六、面试问答准备

### Q1：介绍一下你的项目

我做了一个面向网络系统管理赛项资料的知识库问答系统。起因是我有 4 万字的赛题资料，查一个配置项要翻好几分钟。

实现上是完整的 RAG 链路：文档提取 → 清洗 → 语义切块 → 中文分词 → TF-IDF 检索 → 大模型生成。

**全程只用 Python 标准库**——没有 LangChain、没有向量数据库。这个约束是环境限制逼出来的，但好处是我必须搞懂每个环节的原理。

项目开源在 Gitee 和 GitHub 上，有完整的 README 和测试。

### Q2：为什么不用现成的框架？

一开始是环境装不了 pip。后来发现这个约束反而有价值——为了不用库，我自己实现了中文分词器和 TF-IDF 检索，这让我理解了检索系统的底层机制。

而且后来我手写过 agent 循环之后再看 LangGraph，发现它就是把我写的 while 循环用图表达出来——**知**

道框架在替你做什么，才能判断什么时候不该用它。

### Q3: 你的检索效果怎么样?

我构建了 28 题的人工标注测试集来评测。直接提问的场景 Top-3 召回率 100%，但我故意加了一组改写提问——把"配置 DHCP"问成"怎么让电脑自动获取 IP"——召回率掉到 75%。

分析下来根因是：关键词检索只能匹配字面，"外网"匹配不到"互联网"，"玩游戏"匹配不到"P2P 应用"。

**这是关键词检索的固有局限，调参解决不了，需要引入向量检索做混合检索。**

### Q4: MCP 是什么？和 Function Calling 什么关系？

MCP 是让 AI 应用调用外部工具的开放协议，解决的是  $M \times N$  问题——没有它的话， $N$  个客户端要对接  $M$  个工具就得写  $N \times M$  套代码。

和 Function Calling 的关系是**不同层次**：Function Calling 是模型能力，解决"模型怎么表达我要调工具"；

MCP 是应用协议，解决"AI 应用怎么发现和调用外部工具"。**MCP 是上游**——它把工具接进来，转成工具定义给模型，模型再用 Function Calling 决定调哪个。

我自己用 Python 标准库实现过一个 MCP Server，实现了 initialize / tools/list / tools/call 三个方法。

### Q5: 你怎么调试 agent?

我给 agent 加了埋点，记录每一步的耗时和 token。跑下来发现三个规律：

第一，**工具执行是 0.00 秒，模型调用占了 100% 的时间**——所以性能优化的战场只有一个：减少模型调用轮数。

第二，**输入 token 是输出的 19 倍**，主要来自工具定义——所以工具描述不能写太长，每个字都按输入计费。

第三，**输出 token 数基本决定了生成时间**，大约每秒 50-90 个 token。

这套埋点还帮我抓到一个真 bug：文件工具没展开 ，一跑就报 No such file or directory。

### Q6: 你踩过哪些坑?

印象最深的是 PDF 文本提取。用 pdftotext 抽出来的标题，大概 70% 被"吞"进了上一段。我写正则修复的

时候差点翻车——语料里全是 IP 地址，172.16.100.221；8.8.8.8；5. 标题 如果不加负向断言排除数字，82 个 IP 会被切得七零八落。

还有一个更隐蔽的：pdftotext 在每页边界插入换页符 \f，这个不可见字符被 Python 当成空白字符，让"1 个空格的缩进" 算成了 2 个，触发了我本不该触发的拼接逻辑。 排查了很久，最后靠 hexdump 看原始字节才找到。

Q7：你的项目有什么不足？

- 三个：
- 第一，检索只支持关键词匹配，同义表达召回率只有 75%，需要加向量检索做混合检索。
  - 第二，没有做对话上下文管理。现在的对话历史会一直累积，长对话会超出上下文窗口，需要加压缩策略。
  - 第三，语料本身有局限。我的资料是赛题，只有"任务要求" 没有"配置步骤"，所以问"怎么配"经常答不上来——这个我在 README 里明确写了。

七、文件清单

主项目（~/rag-project/）

|                      |                        |
|----------------------|------------------------|
| rag-project/         |                        |
| └─ README.md         | 项目文档（18 个章节）           |
| └─ build_corpus.py   | ① 将PDF / docx → 纯文本    |
| └─ clean_text.py     | ② 清洗修复：标题错位、目录剔除       |
| └─ chunk_docs.py     | ③ 切块：标题层级 + 面包屑        |
| └─ chunks.json       | 知识块数据（131 块）           |
| └─ retrieval.py      | ④⑤ 分词 + 索引 + TF-IDF 检索 |
| └─ rag.py            | ⑥ 命令行 RAG              |
| └─ eval_retrieval.py | ⑦ 28 题效果评测             |
| └─ mcp_server.py     | ⑧ MCP Server           |

|                |              |
|----------------|--------------|
| └─ test_mcp.py | MCP 测试脚本     |
| └─ api.py      | ⑨ FastAPI 服务 |
| └─ examples/   | 学习示例 (8 个)   |
| └─ docs/       | 技术文档 (2 篇)   |

学习目录 (~/xiangmu/)

|                  |                  |
|------------------|------------------|
| xiangmu/         |                  |
| └─ agent_demo.py | Agent 概念演示       |
| └─ loop_demo.py  | 死循环演示            |
| └─ hello_ai.py   | 调用 LLM API       |
| └─ chat.py       | 对话记忆             |
| └─ agent.py      | 完整 agent (3 个工具) |
| └─ agent_v2.py   | agent + 可观测性     |
| └─ notes.txt     | 学习笔记             |
| └─ jl/           | 参考简历截图           |

交付物

|                                                                                                                     |                |
|---------------------------------------------------------------------------------------------------------------------|----------------|
| ~/张移凡-AI 应用开发-山西电子科技学院.pdf                                                                                          | 简历 (2 页, 带证件照) |
| ~/简历.html                                                                                                           | 简历源文件          |
| ~/mkpdf.sh                                                                                                          | 简历生成脚本         |
| ~/photo.jpg                                                                                                         | 证件照            |
| Gitee: <a href="https://gitee.com/tianxiawo/tianxia">gitee.com/tianxiawo/tianxia</a>                                |                |
| GitHub: <a href="https://github.com/tianxiadiyi20030806-eng/tianxia">github.com/tianxiadiyi20030806-eng/tianxia</a> |                |

八、还没学的

按优先级排序。

| 优先级 | 缺什么        | 为什么重要     | 怎么补                           |
|-----|------------|-----------|-------------------------------|
| ● 高 | 向量检索       | 补上评测发现的短板 | 引入 Embedding + 余弦相似度，做混合检索    |
| ● 高 | 流式输出 (SSE) | 用户体验，面试常问 | FastAPI StreamingResponse     |
| ● 中 | Docker 部署  | 工程化必备     | 写 Dockerfile + docker compose |
| ● 中 | 上下文管理      | 长对话必需     | 实现对话压缩/摘要                     |
| ● 低 | 多 Agent 协作 | 进阶能力      | 规划者 / 执行者 / 检查者角色分工           |
| ● 低 | 安全         | 生产必需      | 工具权限控制、注入防护、沙箱                |

## 附：学习方法总结

### 有效的方法

1. **先跑通，再理解**——看到结果比看懂解释更重要
2. **把新东西翻译成已会的东西**——比如"LangGraph 的合并就是字典的 `update()`"
3. **预测 + 验证**——改代码前先猜结果，跑完对答案
4. **改着跑，不只看**——同样的代码跑十遍没意义，改一点才有新认知

### 一个通用技巧

遇到看不懂的新技术，问自己："这东西用我已经会的东西怎么表达？"

### 心态

觉得难，不代表学不会——往往代表进度快。7 天啃完 Python → LLM API → Agent → RAG → 文档工程 → 中文分词 → 检索算法 → 评测 → MCP 协议 → Web 服务 → 框架，任何人都会觉得难。